

```
/*
 * Programa para listar o conteudo de diretorios Ext2.
 *
 * Cada arquivo passado na linha de comando e' interpretado como o
 * dump de um diretorio.
 *
 * (c) rro, 2016-11-21
 */
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>

/*
 * Estrutura de um diretorio no ext2
 * http://www.nongnu.org/ext2-doc/ext2.html#DIRECTORY
 */

struct dirent_ext2 {
    uint32_t inode;
    uint16_t reclen;
    uint8_t name_len;
    uint8_t file_type;
    unsigned char name[256];
};

/* mnemonicos para tipos de arquivos */
char *ftypes[] = { "desc", "reg ", "dir ", "char", "blk ", "fifo", "sock",
                  "syml" };

int main(int argc, char *argv[])
{
    FILE *fp;
    int i, j;
    size_t pos;
    struct dirent_ext2 de;

    if (argc < 2) {
        fprintf(stderr, "uso: %s arq1 [arq2 ... arqN]\n", argv[0]);
        fprintf(stderr, "onde arq1, arq2, ..., arqN sao dumps de blocos "
            "contendo diretorios\n");
        exit(1);
    }
    for (i=1; i < argc; i++) {
        fp = fopen(argv[i], "r");
        if (fp == NULL) {
            fprintf(stderr, "%s: erro na abertura de %s\n\n", argv[0],
                argv[i]);
            continue; /* avanca para o proximo arquivo */
        }
        printf("%s:\n", argv[i]);
        j = 0; /* contador sequencial de entradas */
        printf(" # inode reclen name_len file_type name\n");
        while (!feof(fp)) {
            pos = ftell(fp); /* salva posicao no arquivo */
            if (fread(&de.inode, sizeof(de.inode), 1, fp) == 0) {
                if (pos == 0) {
                    fprintf(stderr,
                        "%s: erro na leitura de %s (inode curto)\n",
                        argv[0], argv[i]);
                }
                break; /* processa proximo arquivo */
            }
            if (fread(&de.reclen, sizeof(de.reclen), 1, fp) == 0) {
                fprintf(stderr,
                    "%s: erro na leitura de %s (reclen curto)\n",
                    argv[0], argv[i]);
                break; /* processa proximo arquivo */
            }
        }
    }
}
```

```
if (fread(&de.name_len, sizeof(de.name_len), 1, fp) == 0) {
    fprintf(stderr,
        "%s: erro na leitura de %s (name_len curto)\n",
        argv[0], argv[i]);
    break; /* processa proximo arquivo */
}
if (fread(&de.file_type, sizeof(de.file_type), 1, fp) == 0) {
    fprintf(stderr,
        "%s: erro na leitura de %s (file_type curto)\n",
        argv[0], argv[i]);
    break; /* processa proximo arquivo */
}
if (fread(&de.name, de.name_len, 1, fp) == 0) {
    fprintf(stderr,
        "%s: erro na leitura de %s (name curto)\n",
        argv[0], argv[i]);
    break; /* processa proximo arquivo */
}
/* termina string */
de.name[de.name_len] = '\0';

/* imprime entrada */
printf(" %03d: %5u %6u %8u %4u/%s %s\n", j, de.inode,
    de.reclen, de.name_len, de.file_type,
    ftypes[de.file_type], de.name);

/* posiciona na proxima entrada */
fseek(fp, pos + de.reclen, SEEK_SET);
j++;
}
putchar('\n');
}
return 0;
}
```